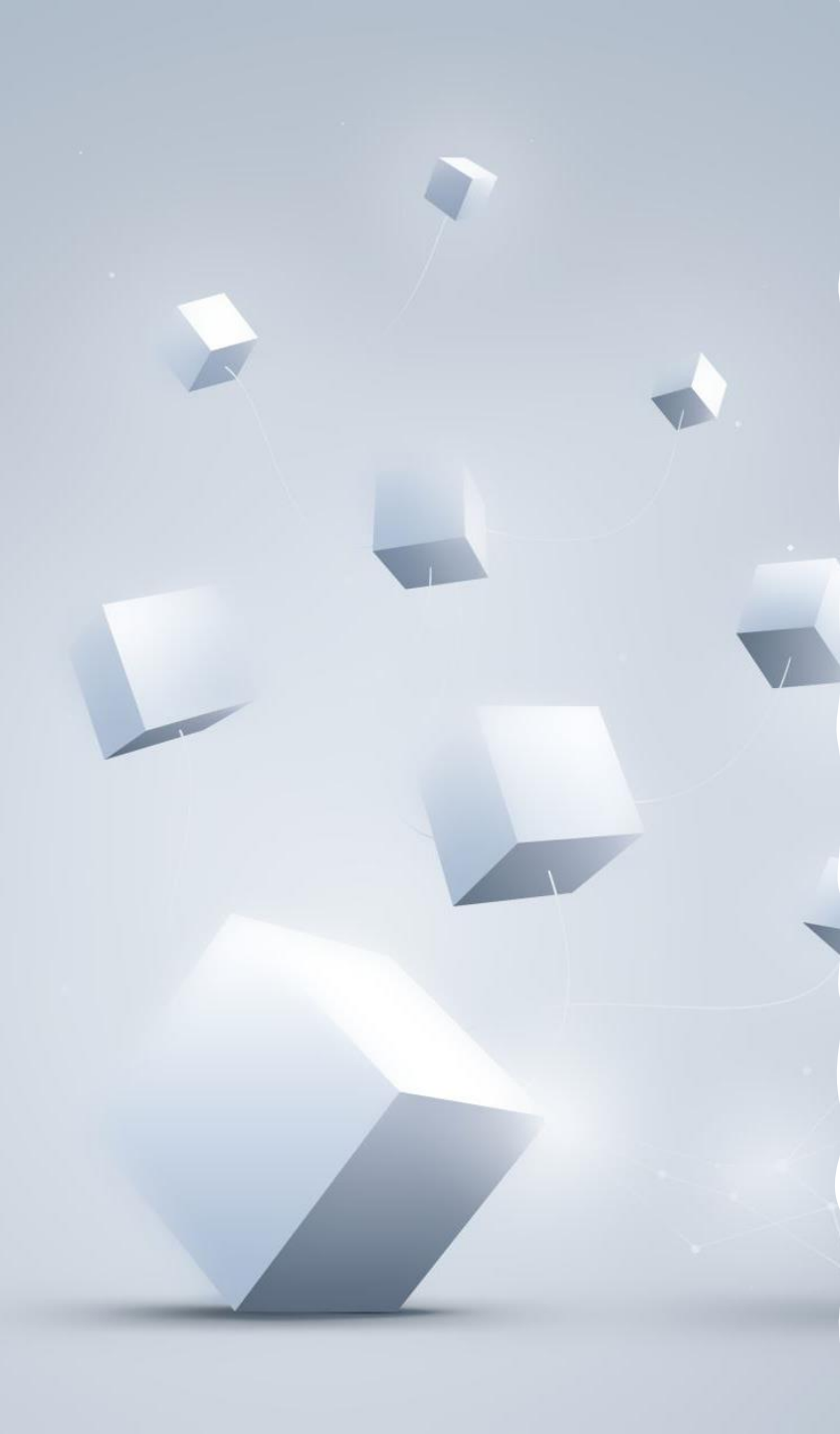
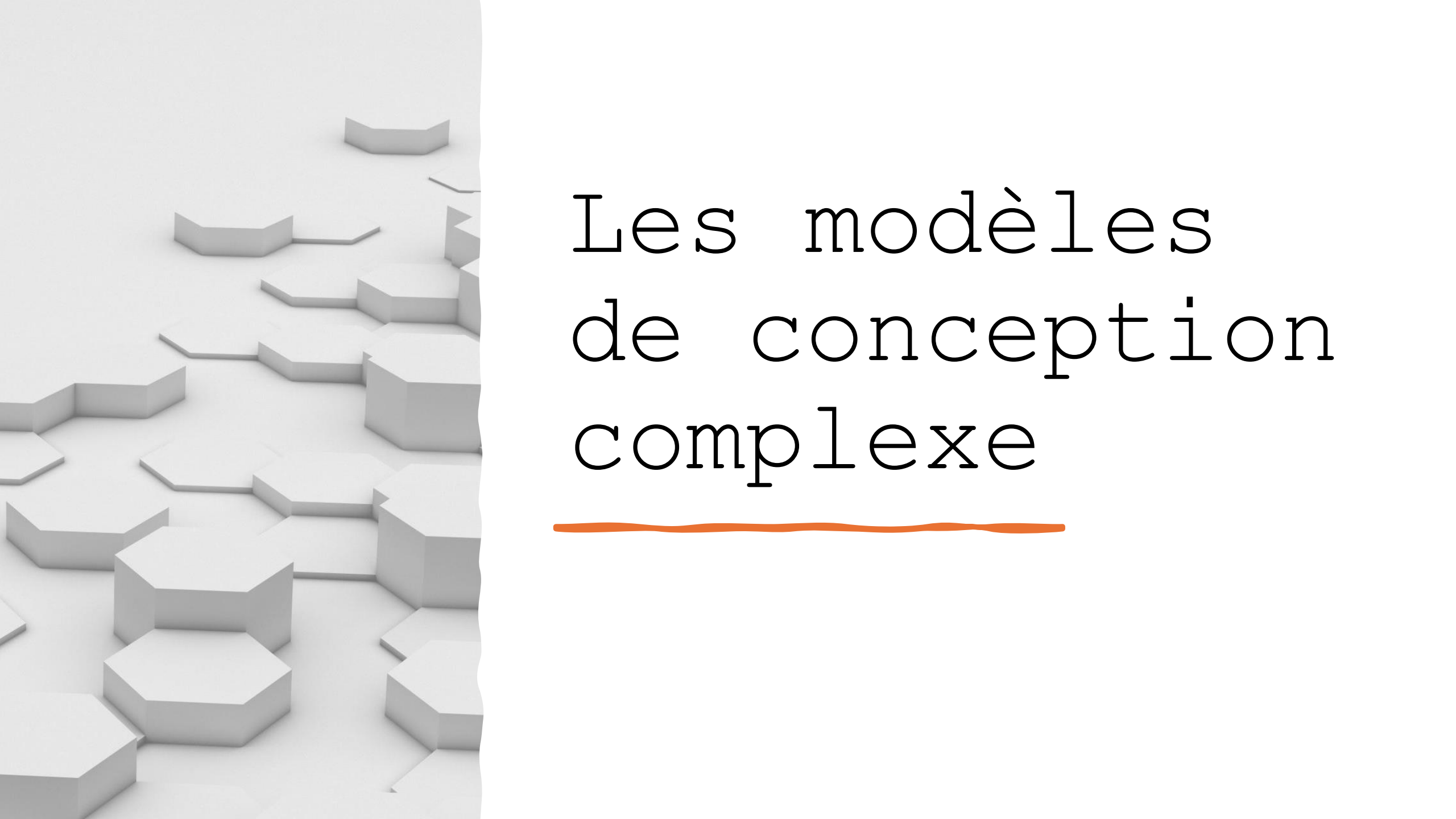




Modele Complexe

Dr Mamadou Bousso



Les modèles de conception complexe

Catalogue des modèles de conception complexes



Fabrique abstraite (Abstract Factory) : fournit une interface pour fabriquer des familles d'objets apparentés ou dépendants sans avoir à spécifier leurs classes concrètes; (**Usine**)



Adaptateur (Adapter) : convertit l'interface d'une classe en une interface distincte, conforme à l'attente de l'utilisateur. Il permet à des classes de travailler ensemble; (**Multiprise**)



Pont (Bridge) : découple une abstraction de son implémentation associée afin que les deux puissent être modifiés indépendamment. (**télécommande et téléviseur**)



Monteur (Builder) : Dans un objet complexe, dissocie sa construction de sa représentation, de sorte que, le même procédé de construction puisse engendrer des représentations différentes.

Catalogue des modèles de conception



Chaine de responsabilité (Chain of responsibility) : permet d'éviter de coupler l'expéditeur d'une requête à son destinataire, en donnant la possibilité à plusieurs objets de prendre en charge la requête. Pour ce faire, il chaine les objets récepteurs, et fait passer la requête tout au long de cette chaine jusqu'à ce qu'un des objets la prenne en charge. **(gestion des emails)**



Commande (Command) : encapsule une requête comme un objet, ce qui permet de faire un paramétrage des clients avec différentes requêtes; files d'attente, historiques de requêtes et d'assurer le traitement des opérations réversibles **(Action, transaction)**.



Composite : organise les objets en structure arborescente représentant la hiérarchie de haut en bas. Il permet aux utilisateurs de traiter des objets individuels et des ensembles organisés de ces objets de la même manière.



Catalogue des modèle de conception

- **Décorateur (Decorator)** : attache des responsabilités à un objet de façon dynamique. Alternative à la dérivation des classes pour l'extension des fonctionnalités. (**Prix des pizzas**)
- **Façade (Facade)** : fournit une interface unifiée pour un ensemble d'interfaces d'un sous-système. Elle définit une interface de haut niveau qui rend le sous-système plus facile à utiliser. (**JEE**)
- **Fabrication (Factory method)** : définit une interface pour la création d'un objet, tout en laissant à des sous-classes le choix de la classe à instancier.
- **Poids mouche (Flyweight)** : Assure en mode partagé le support efficace d'un grand nombre d'objets à fine granularité.

Catalogue des modèle de conception



Interprète (Interpreter) : dans un langage donné, il définit une représentation de sa grammaire, ainsi qu'un interprète qui utilise cette représentation pour analyser la syntaxe du langage.



Iterateur (Iterator) : fournit un moyen pour accéder en séquence aux éléments d'un objet de type agrégat sans révéler sa représentation sous-jacente.



Mediateur (Mediator) : définit un objet qui encapsule les modalités d'interaction de divers objets. Il favorise les couplages faibles, en dispensant les objets d'avoir à faire une référence explicite les uns aux autres.



Procuration (Proxy) : fournit un remplaçant d'un autre objet pour en contrôler l'accès.

Catalogue des modèle de conception



Memento (Memento) : sans violer l'encapsulation, acquiert et délivre des informations sur l'état interne d'un objet afin que celui-ci puisse être rétabli ultérieurement dans cet état. (**requête undo**)



Observateur (Observer) : définit une corrélation entre objets du type un à plusieurs, lorsqu'un objet change d'état, tous ceux qui en dépendent en seront notifiés et mis à jour automatiquement.



Prototype (Prototype) : spécifie les espèces d'objets à créer, en utilisant une instance de type prototype, et crée de nouveaux objets par copie de ce prototype.



Singleton (Singleton) : garantit qu'une classe n'a qu'une seule instance, et fournit à celle-ci, un point d'accès de type global.

Catalogue des modèle de conception

Etat (State) : permet à un objet de modifier son comportement lorsque son état interne change. L'objet paraîtra changer de classe.

Stratégie (Strategy) : définit une famille d'algorithmes, encapsule chacun d'entre eux, et les rend interchangeables. Elle permet de modifier un algorithme indépendamment de ses clients.

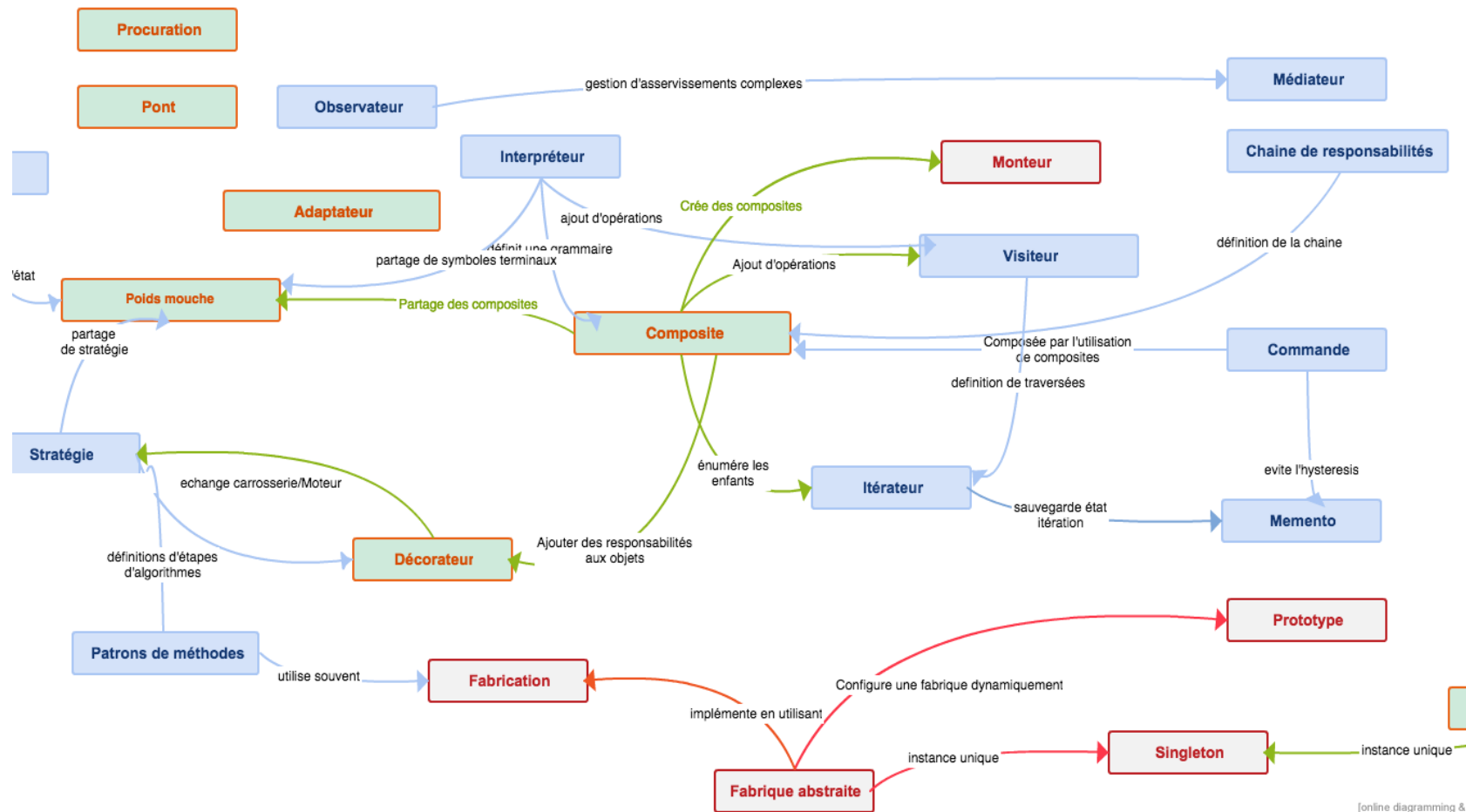
Patron de méthodes (Template method) : définit le squelette de l'algorithme d'une opération, en déléguant le traitement de certaines étapes à des sous-classes. Il permet aux sous-classes de redéfinir certaines étapes d'un algorithme sans modifier sa structure.

Visiteur (Visitor) : représente une opération à effectuer sur les éléments d'une structure d'objet. Il ne modifie pas les classes des éléments sur lesquels il opère.

Organisation du catalogue

		ROLE		
		CREATEUR	STRUCTUREL	COMPORTEMENTAL
DOMAINE	CLASSE	Fabrication	Adaptateur	Interprète
				Patron de méthode
	OBJET	Fabrique abstraite	Adaptateur	Chaine de responsabilité
		Monteur	Pont	Commande
		Prototype	Composite	Itérateur
		Singleton	Décorateur	Médiateur
			Facade	Memento
			Poids Mouche	Observateur
			Procuration	Etat
				Stratégie
				Visiteur

Relations entre modèle de conception



[online diagramming & de

Quelques solutions apportées par les modèles de conception

- *Créer un objet en spécifiant spécifiquement une classe* complique les modifications futures
 - **Solution:** créer les objets indirectement avec les modèles Fabrique, Fabrique abstraite, Prototype
- *Assujettissement à une opération particulière...* requête codée en dur
 - **Modèles solution:** Chaine de responsabilités, commandes

Quelques solutions apportées par les modèles de conception

- *Dépendance vis à vis des plateforme matérielles et logicielles ...logiciels peu portable, maintenance difficile*
 - **Solution:** les modèles Fabrique abstraite et Pont
- *Assujettissement à la représentation d'un objet ou à son code ... Modification en cascade lorsque l'objet change,*
 - **Solution:** cacher la représentation des objets au client avec Fabrique abstraite, Pont, Memento; Procuration

Quelques solutions apportées par les modèles de conception

- *Assujettissement à un algorithme ...changement perpétuel des objets dépendants*
 - **Solution:** Monteur, itérateur, patron de méthodes, Stratégie, visiteur
- *Couplage fort ... Classes difficilement utilisables séparément, systèmes denses difficiles à étudier et à maintenir*
 - **Solution:** implanter des systèmes à couplage faible avec Fabrique abstraite, Pont, Médiateur, Observateur, Façade, Chaîne de responsabilité

Quelques solutions apportées par les modèles de conception

- *Extension de fonctionnalités par sous-classes*
...opérations délicates qui peut conduire à une prolifération de sous-classes
 - **Solution:** Opter pour la délégation et la composition avec les modèles Pont, Observateur, Chaine de responsabilité, Composite, Stratégie.
- *Impossibilité de modifier convenablement les classes à cause d'éventuelles modifications en cascade*
 - **Solution:** modifier les classes avec les modèles Adaptateur, Décorateur, Visiteur

DESIGN PATTERN- Introduction



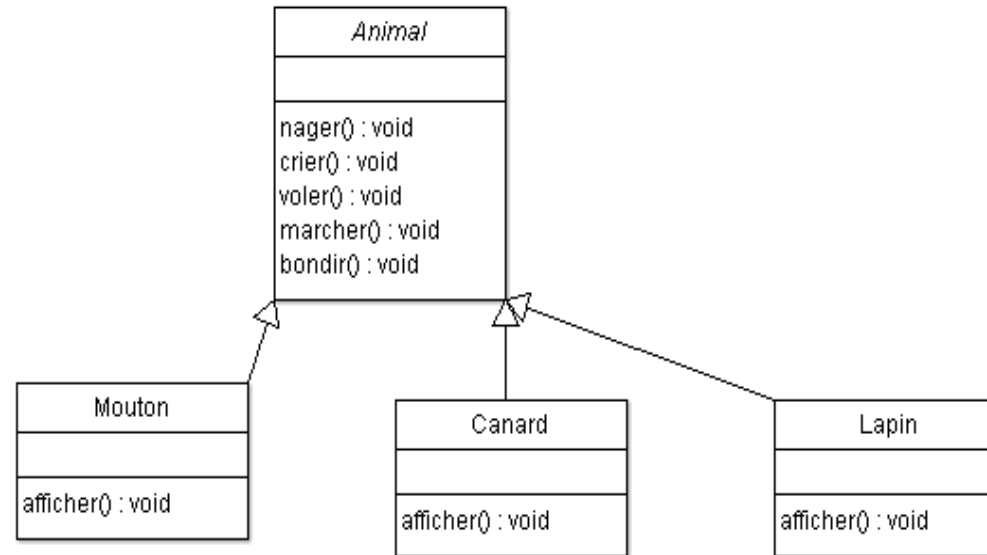
DESIGN PATTERN- Introduction

Exercice:

On désire réaliser un programme java qui simule les cris et mode de déplacement des animaux du transparent précédent. Imaginer un design simple basée sur une technique orientée objet et permettant de modéliser ces animaux

DESIGN PATTERN- Introduction

Modou réalise le design suivant:



DESIGN PATTERN- Introduction

Exercice:

Quel sont les inconvenients de ce design ?

DESIGN PATTERN- Introduction



Le code est dupliqué à travers toutes les sous-classes

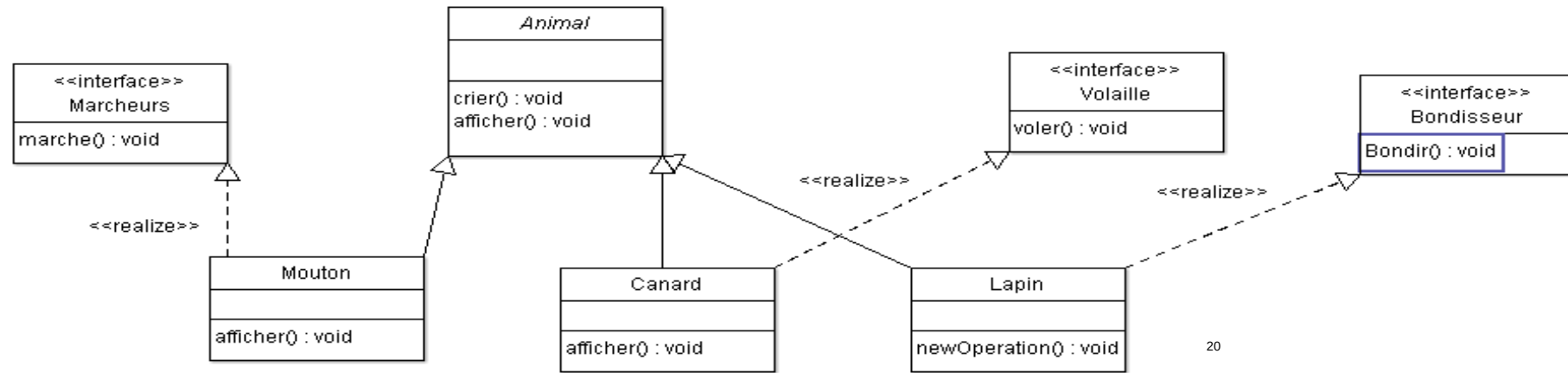


Le mouton ne vole pas mais implémente une méthode voler()



Tout changement sur la classe de base affecte toutes les sous-classes

- Doudou propose un autre design basé sur les interfaces



20

DESIGN PATTERN- Introduction

- Que pensez-vous du design de Doudou ?
- Qu'allons nous faire si nous voulons ajouter les classes poisson, coq et corbeau ?

DESIGN PATTERN- Introduction

Le design de Modou est mauvais car le code est répliqué dans toutes les sous-classes

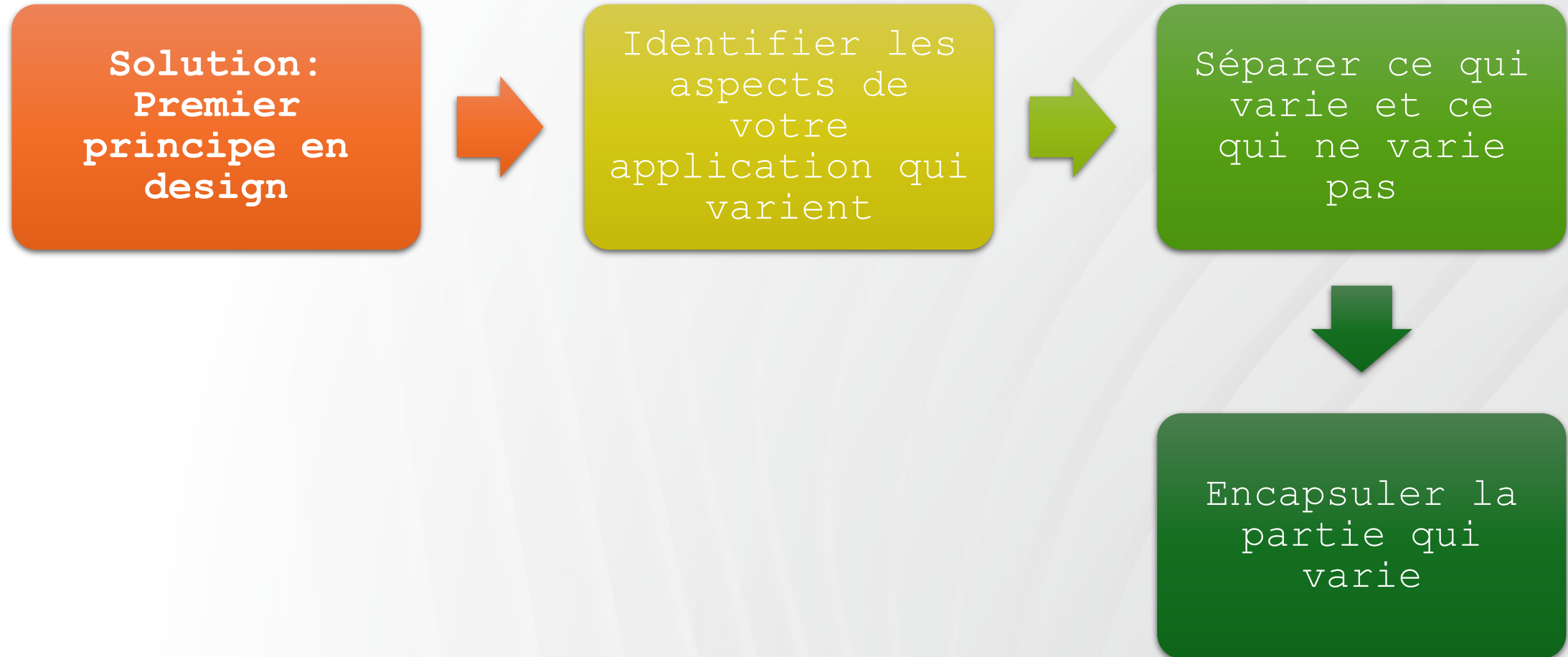
Ceci est partiellement contourné dans le design de Doudou

Mais il va falloir implémenter des méthodes dans toutes les sous-classes implémentant les interfaces;

Définir de nouvelles interfaces à chaque fois qu'il y a de nouveaux comportements;

S'il y en a 1000 c'est la catastrophe

DESIGN PATTERN- Introduction



DESIGN PATTERN- Introduction

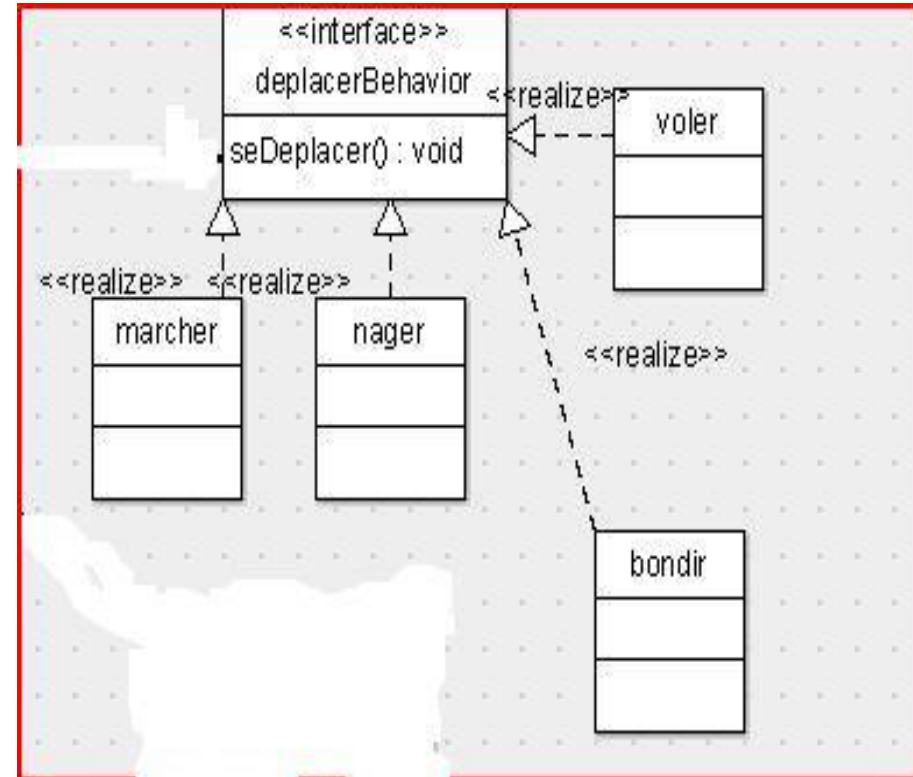
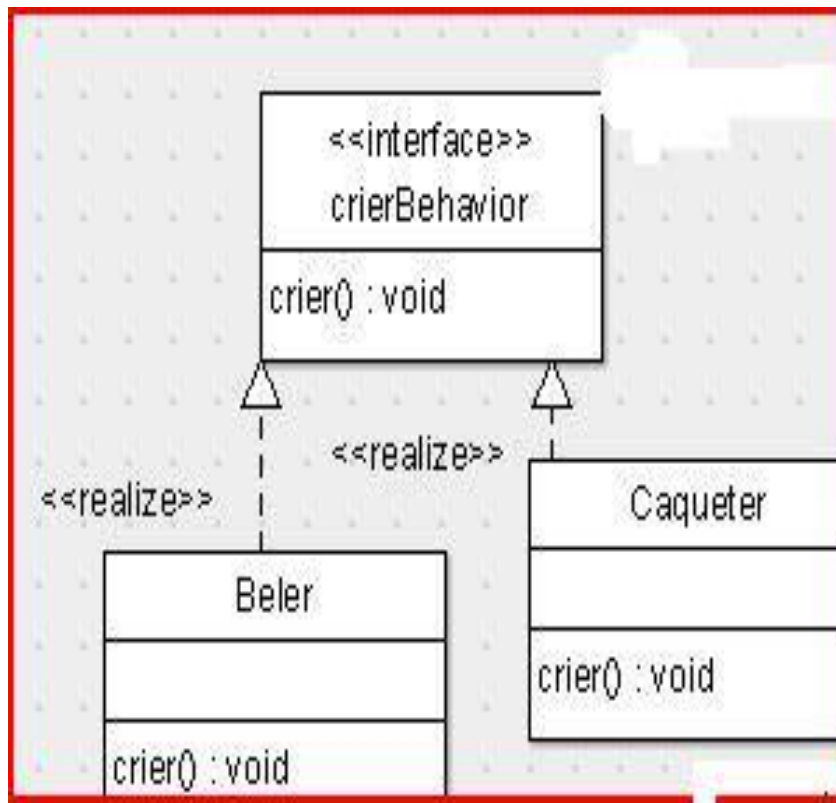
Exercice:

Identifier
la partie
qui variera
dans le code

Encapsuler
la partie
qui varie

DESIGN PATTERN- Introduction

Délégation: La classe délègue ses comportements variables à d'autres classes



DESIGN PATTERN- Introduction

- Avec ce nouveau design d'autres objets peuvent utiliser les comportements `seDeplacer()` et `crier()`
- Nous pouvons ajouter d'autres comportements sans modifier ceux existant

DESIGN PATTERN-

Introduction

- ❑ La classe **Animal** a délégué ses comportements **seDeplacer()** et **crier()**
- ❑ Nous allons lui ajouter deux variables d'instance **DepBehavior** et **CriBehavior** du même type que les classes déléguées
- ❑ Nous allons lui ajouter deux méthodes **faireCri()** et **faireMouvement()**

DESIGN PATTERN- Introduction

- ❑ Pour crier la classe Animal permet à l'objet **CriBehavior** de crier dans la méthode faireCri()
- ❑ Le type de l'objet ne nous intéresse pas.
- ❑ L'intérêt est qu'il sache crier

DESIGN PATTERN- Introduction

Exercice:

- ❑ Ecrire le nouveau code de la classe Animal
- ❑ Ecrivez le code associé à la classe Mouton particulièrement son constructeur
- ❑ Testez les différentes classes definies ici

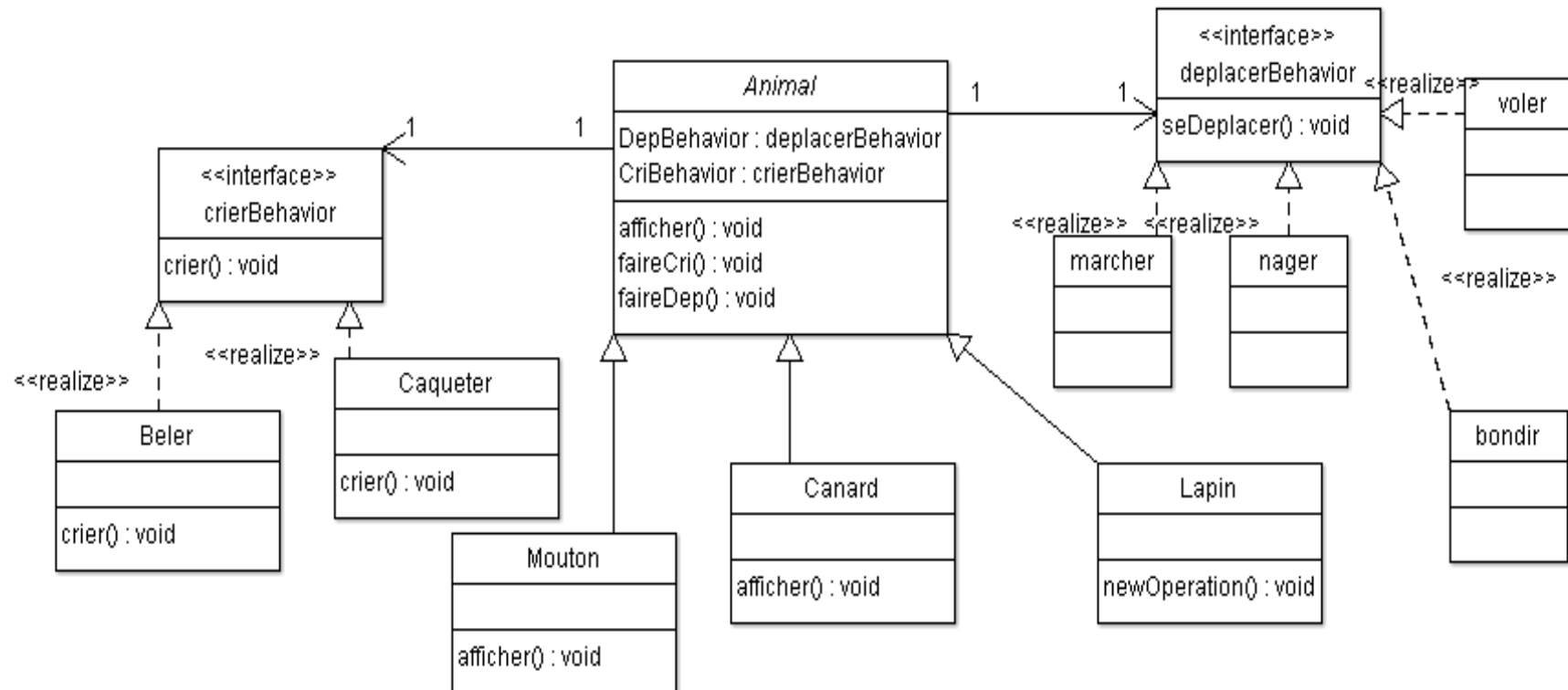
DESIGN PATTERN- Introduction

- Ajoutons un comportement dynamique avec les méthodes suivantes:

```
Public void setCriBehavior(CriBehavior cb){  
    CriBehavior = cb;  
}  
Public void setDepBehavior(DepBehavior db){  
    DepBehavior = db;  
}
```

DESIGN PATTERN- Introduction

- Nous obtenons le diagramme suivant avec les relations EST-UN, A-UN, IMPLEMENTE



DESIGN PATTERN-

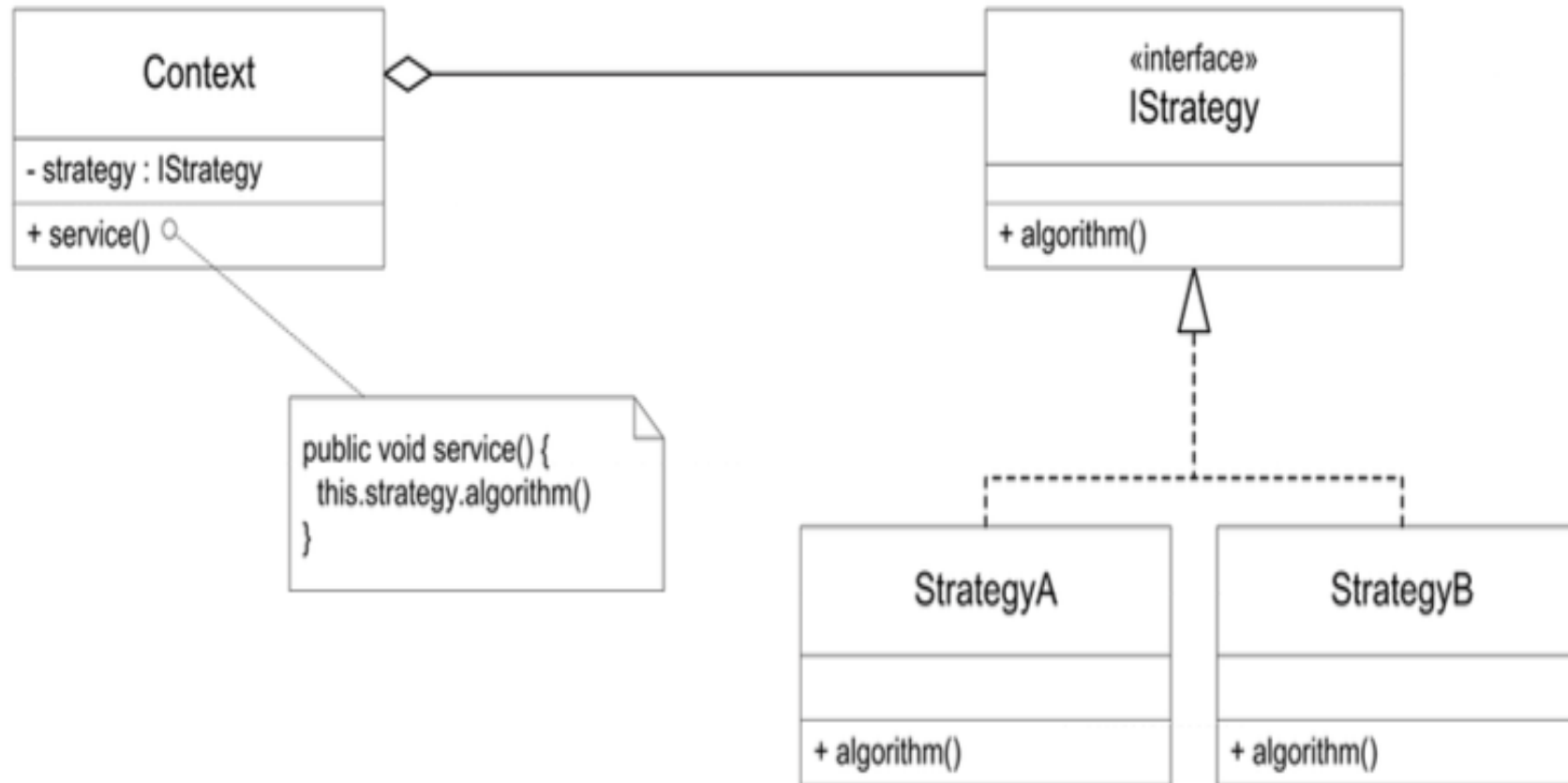
Introduction

- La relation A-UN est intéressante car elle permet à chaque animal d'avoir un cri et de se déplacer
- La classe animal a un nouveau comportement grâce à une **composition** avec un objet définissant ce comportement
- **Deuxième principe: Favoriser la composition sur l'héritage**

DESIGN PATTERN- Introduction

- ❑ Ce pattern est appelé **strategy pattern**
- ❑ Il définit une famille d'algorithmes encapsulés chacune, et les rend interchangeable
- ❑ Ces algorithmes varient indépendamment du client qui les utilisent

DESIGN PATTERN- Introduction





FIN